Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-211БВ-25

Студент: Шеменеева А. Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 18.10.26

Москва, 2026

Постановка задачи

Вариант 4.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Отсортировать массив целых чисел при помощи TimSort.

Использованные системные вызовы:

- pthread_mutex_lock() - это функция, реализующая операцию захвата мьютекса.
- pthread_cond_wait() - переводит поток в состояние ожидание, освобождает мьютекс.
- pthread_cond_signal() - функция, реализующая сигнальное пробуждение одного потока.
- pthread_mutex_unlock() - освобождает мьютекс.
- pthread_mutex_init(), pthread_cond_init() - инициализация мьютекса и условных переменных.
- pthread_create() - создает поток.
- pthread_detach() - переводит потомк в отсоединенный режим.
- pthread_cond_broadcast - сигнальное пробеждение всех потоков.

Запуск программы, инициализация, выделение памяти

execve("./program", ["./program", "100000", "10", "par"], 0x7ffd10fd7dc8 /* 26 vars */) = 0

brk(NULL) = 0x5f209cf06000

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7b105fa27000

access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=21959, ...}) = 0

mmap(NULL, 21959, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7b105fa21000

close(3)

Создание 10ти потоков

write(1, "\320\237\320\220\320\240\320\220\320\233\320\233\320\225\320\233\320\254\320\235\320\253\320\231\n", 25) = 25

write(1, "\320\227\320\260\320\277\321\200\320\276\321\210\320\265\320\275\320\276\320\277\320\276\321\202\320\276\320\272\320\276\320"..., 38) = 38

rt_sigaction(SIGRT_1, {sa_handler=0x7b105f899610, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7b105f845330}, NULL, 8) = 0

rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7b105ef9d000

mprotect(0x7b105ef9e000, 8388608, PROT_READ|PROT_WRITE) = 0

rt_sigprocmask(SIG_BLOCK, ~[], [], 8)   = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7b105f79d990, parent_tid=0x7b105f79d990, exit_signal=0, stack=0x7b105ef9d000, stack_size=0x7fff80, tls=0x7b105f79d6c0} => {parent_tid=[2782]}, 88) = 2782

.....

rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7b105a794000

mprotect(0x7b105a795000, 8388608, PROT_READ|PROT_WRITE) = 0

rt_sigprocmask(SIG_BLOCK, ~[], [], 8)   = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7b105af94990, parent_tid=0x7b105af94990, exit_signal=0, stack=0x7b105a794000, stack_size=0x7fff80, tls=0x7b105af946c0} => {parent_tid=[2791]}, 88) = 2791

rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

Пробуждение потоков, мьютексы

futex(0x5f209cf06724, FUTEX_WAKE_PRIVATE, 1) = 1

futex(0x5f209cf066d0, FUTEX_WAKE_PRIVATE, 1) = 1

Поток ожидает на мьютексе

futex(0x5f209cf06780, FUTEX_WAIT_BITSET_PRIVATE|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY) = 0

pthread_cond_broadcast

futex(0x5f209cf06720, FUTEX_WAKE_PRIVATE, 2147483647) = 5

futex(0x5f209cf06724, FUTEX_WAKE_PRIVATE, 2147483647) = 5

Завершение программы, вывод результат

write(1, "\n", 1)   = 1

write(1, "\320\222\321\200\320\265\320\274\321\217
\320\262\321\213\320\277\320\276\320\273\320\275\320\265\320\275\320\270\321\217:"..., 48) = 48

write(1, "\320\234\320\260\321\201\321\201\320\270\320\262
\320\276\321\202\321\201\320\276\321\200\321\202\320\270\321\200\320\276\320"..., 44) = 44

munmap(0x7b105f79e000, 401408)   = 0

exit_group(0)   = ?

+++ exited with 0 +++

Код программы

timsort.c

```
#include <pthread.h> #include <stdlib.h> #include <stdio.h> #include <string.h>
#include <time.h> #include <unistd.h>

void insertion_sort(int *arr, int l, int r) { for (int i = l + 1; i <= r; i++) { int
tmp = arr[i]; int j = i - 1;

    while (j >= l && arr[j] > tmp){
    arr[j + 1] = arr[j];
    j--;
}
arr[j + 1] = tmp;
}


}

void merge(int *arr, int l, int m, int r) { int len1 = m - l + 1; int len2 = r - m;
int *L = (int)malloc(len1 * sizeof(int)); int *R = (int)malloc(len2 * sizeof(int));

for (int i = 0; i < len1; i++) L[i] = arr[l + i];
for (int j = 0; j < len2; j++) R[j] = arr[m + 1 + j];

int i = 0, j = 0, k = l;
while (i < len1 && j < len2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}
while (i < len1) arr[k++] = L[i++];
while (j < len2) arr[k++] = R[j++];

free(L);
free(R);


}

void timsort_seq(int *arr, int n) { int run_size = 32;

for (int i = 0; i < n; i += run_size) {
    int right = (i + run_size - 1 < n - 1) ? (i + run_size - 1) : (n - 1);
    insertion_sort(arr, i, right);
}
for (int size = run_size; size < n; size = 2 * size) {
    for (int left = 0; left < n; left += 2 * size) {
        int mid = left + size - 1;
        int right = (left + 2 * size - 1 < n - 1) ? (left + 2 * size - 1) : (n - 1);
        if (mid < right) {
            merge(arr, left, mid, right);
        }
    }
}
```

```
    }

    typedef struct { int *array; int left; int mid; int right; } Task;

    typedef struct { Task *tasks; int capacity; int head; int tail; int count; int
active_threads;

    pthread_mutex_t lock;
pthread_cond_t not_empty;
pthread_cond_t not_full;
pthread_cond_t all_done;

int shutdown;


    } ThreadPool;

    void* worker_thread(void* arg) { ThreadPool pool = (ThreadPool)arg;

    while(1) {
    pthread_mutex_lock(&pool->lock);
    while (pool->count == 0 && !pool->shutdown) {
        pthread_cond_wait(&pool->not_empty, &pool->lock);
    }
    if (pool->shutdown && pool->count == 0) {
        pthread_mutex_unlock(&pool->lock);
        break;
    }

    Task task = pool->tasks[pool->head];
    pool->head = (pool->head + 1) % pool->capacity;
    pool->count--;
    pool->active_threads++;

    pthread_cond_signal(&pool->not_full);
    pthread_mutex_unlock(&pool->lock);

    merge(task.array, task.left, task.mid, task.right);

    pthread_mutex_lock(&pool->lock);
    pool->active_threads--;
    if (pool->active_threads == 0 && pool->count == 0) {
        pthread_cond_signal(&pool->all_done);
    }
    pthread_mutex_unlock(&pool->lock);
}
return NULL;


    }

    void print_os_threads() { char command[256]; sprintf(command, "ps -o nlwp= -p %d",
getpid()); printf("Количество потоков в процессе сейчас: "); fflush(stdout);
system(command); }

    ThreadPool create_pool(int num_threads, int queue_capacity) { ThreadPool pool =
(ThreadPool)malloc(sizeof(ThreadPool)); pool->tasks = (Task)malloc(queue_capacity *
sizeof(Task)); pool->capacity = queue_capacity; pool->head = 0; pool->tail = 0; pool->count
= 0; pool->active_threads = 0; pool->shutdown = 0;

    pthread_mutex_init(&pool->lock, NULL);
pthread_cond_init(&pool->not_empty, NULL);
pthread_cond_init(&pool->not_full, NULL);
pthread_cond_init(&pool->all_done, NULL);

pthread_t *threads = (pthread_t*)malloc(num_threads * sizeof(pthread_t));
for (int i = 0; i < num_threads; i++) {
    pthread_create(&threads[i], NULL, worker_thread, pool);
    pthread_detach(threads[i]);
}

print_os_threads();

free(threads);
return pool;


    }

    void add_task(ThreadPool *pool, int *arr, int l, int m, int r)
{ pthread_mutex_lock(&pool->lock); while (pool->count == pool->capacity && !pool->shutdown)
{ pthread_cond_wait(&pool->not_full, &pool->lock); } pool->tasks[pool->tail].array = arr;
pool->tasks[pool->tail].left = l; pool->tasks[pool->tail].mid = m; pool->tasks[pool-
>tail].right = r; pool->tail = (pool->tail + 1) % pool->capacity; pool->count++;
pthread_cond_signal(&pool->not_empty); pthread_mutex_unlock(&pool->lock); }

    void wait_for_completion(ThreadPool *pool) { pthread_mutex_lock(&pool->lock); while
(pool->count > 0 || pool->active_threads > 0) { pthread_cond_wait(&pool->all_done, &pool-
>lock); } pthread_mutex_unlock(&pool->lock); }

    void destroy_pool(ThreadPool *pool) { pthread_mutex_lock(&pool->lock); pool->shutdown
= 1; pthread_cond_broadcast(&pool->not_empty); pthread_mutex_unlock(&pool->lock); free(pool-
>tasks); free(pool); }

    void timsort_parallel(int *arr, int n, int num_threads) { int runs_size = 32; for (int
i = 0; i < n; i += runs_size) { int right = (i + runs_size - 1 < n -1) ? (i + runs_size -
1) : (n - 1); insertion_sort(arr,i, right); }

    ThreadPool *pool = create_pool(num_threads, 1024);

for (int size = runs_size; size < n; size = 2*size) {
    for (int l = 0; l < n; l += 2*size) {
        int m = l + size - 1;
        int r = (l + 2*size - 1 < n - 1) ? (l + 2*size -1) : (n -1);

        if(m < r) add_task(pool, arr, l, m, r);
    }
    wait_for_completion(pool);
```

```
}
destroy_pool(pool);

    }

    double get_time() { struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts); return
ts.tv_sec + ts.tv_nsec / 1e9; }

    int is_sorted(int *arr, int n) { for (int i = 1; i < n; i++) { if (arr[i] < arr[i-1])
return 0; } return 1; }

    int main(int argc, char *argv[]) { if (argc != 4) { printf("\n"); }

    int n = atoi(argv[1]);
int num_threads = atoi(argv[2]);
char *mode = argv[3];

if (n <= 0) {
    printf("размер массива должен быть > 0\n");
    return 1;
}

int *arr = (int*)malloc(n * sizeof(int));
srand(time(NULL));
for (int i = 0; i < n; i++) {
    arr[i] = rand() % 1000000;
}

double start_time = get_time();

 if (strcmp(mode, "seq") == 0) {
    printf("ПОСЛЕДОВАТЕЛЬНЫЙ\n");
    timsort_seq(arr, n);
} else if (strcmp(mode,"par") == 0) {
    printf("ПАРАЛЛЕЛЬНЫЙ\n");
    printf("Запрошено потоков: %d\n", num_threads);
    timsort_parallel(arr, n, num_threads);
} else {
    printf("Неизвестный режим: %s\n", mode);
    free(arr);
    return 1;
}

double end_time = get_time();
double duration = end_time - start_time;

int sorted = is_sorted(arr, n);

printf("\n");
printf("Время выполнения: %.4f сек.\n", duration);
printf("Массив отсортирован: %s\n", sorted ? "ДА" : "НЕТ");

free(arr);
return 0;

    }
```

Протокол работы программы

- **Ускорение:** $S = T_s / T_p$, где
  - T_s – время последовательной реализации,
  - T_p – время параллельной реализации,
  - $1 <= S <= p$,
  - p – количество ядер

- **Эффективность**: $X = S / p$, где $X < 1$

Количество логических ядер 12.

Таблица для ввода 100000:

Число потоков	Время исполнения (с)	Ускорение	Эффективность
последовательный	0.0226	-	-
2	0.0413	0,547215496	0,273607748
7	0.0239	0,945606695	0,135086671

11	0.0230	0,982608696	0,089328063
12	0.0338	0,668639053	0,055719921
17	0.0362	0,624309392	0,036724082
129	0.1352	0,167159763	0,001295812
1025	0.2007	0,112605879	0,000109859

Таблица для ввода 100000000:

Число потоков	Время исполнения (с)	Ускорение	Эффективность
последовательный	22.5215	-	-
2	26.9650	0,835212312	0,417606156
7	10.3081	2,184835227	0,312119318
11	9.9711	2,258677578	0,205334325
12	10.1175	2,225994564	0,185499547
17	11.2114	2,008803539	0,118164914
129	-	-	-
1025	-	-	-

Тестирование:

./program 100000 1 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 1
Количество потоков в процессе сейчас: 2

Время выполнения: 0.0413 сек.
Массив отсортирован: ДА

./program 100000 6 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 6
Количество потоков в процессе сейчас: 7

Время выполнения: 0.0239 сек.
Массив отсортирован: ДА

./program 100000 10 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 10
Количество потоков в процессе сейчас: 11

Время выполнения: 0.0230 сек.
Массив отсортирован: ДА

./program 100000 11 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 11
Количество потоков в процессе сейчас: 12

Время выполнения: 0.0338 сек.
Массив отсортирован: ДА

./program 100000 12 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 12
Количество потоков в процессе сейчас: 13

Время выполнения: 0.0419 сек.
Массив отсортирован: ДА

./program 100000 16 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 16
Количество потоков в процессе сейчас: 17

Время выполнения: 0.0362 сек.
Массив отсортирован: ДА

./program 100000 128 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 128
Количество потоков в процессе сейчас: 129

Время выполнения: 0.1352 сек.
Массив отсортирован: ДА

./program 100000 1024 par
ПАРАЛЛЕЛЬНЫЙ
Запрошено потоков: 1024
Количество потоков в процессе сейчас: 1025

Время выполнения: 0.2007 сек.
Массив отсортирован: ДА

./program 100000 12 seq
ПОСЛЕДОВАТЕЛЬНЫЙ

Время выполнения: 0.0226 сек.
Массив отсортирован: ДА

Вывод

В ходе работы была разработана последовательная и параллельная реализации алгоритма Timsort. Параллельная версия использует пул потоков с ограничением, задаваемым аргументом командной строки, и распараллеливает этап слияния (merge) через очередь задач. Наличие потоков подтверждено средствами ОС (ps, трассировка clone3 и futex). Эксперименты показали, что параллельная версия дает ускорение на больших массивах. Максимальная эффективность достигается при количестве потоков, равному 11 логических ядер; при превышении этого числа производительность падает из-за переключений контекста и борьбы за кэш. Цель работы достигнута: получены навыки управления потоками и их синхронизации.